XDRIVE FULL PLATFORM FORENSIC AUDIT - PHASE 2

Repository: LoadifyMarketLTD/xdrivelogistics.co.uk

Deliverable: XDRIVE_FULL_PLATFORM_FORENSIC_AUDIT_PHASE_2_WITH_REPAIR_PLAN.pdf

Method: static repository audit only. No runtime/live verification is claimed unless explicitly stated.

Runtime status for this report: NOT RUNTIME TESTED.

Sandbox limitation statement:

This audit verifies repository code, migration chain and local build health only. It does not verify live Supabase data, Netlify runtime secrets, production browser behavior, storage object contents, or production RLS state beyond what is represented in source and previously recorded context.

1. EXECUTIVE VERDICT

- Current repository state is materially healthier than the earlier static report implied: after npm install, local npm run typecheck, npm run lint and npm run build all succeeded in the sandbox.
- The previously reported 4506 TypeScript errors do not reproduce in the current checked-out repository state; that strongly suggests an older branch state, missing dependencies, or earlier environment mismatch rather than a current repo-wide TypeScript collapse.
- The platform is still NOT ready to be treated as a clean production-grade system of record because the core auth/company/membership isolation model still contains repair logic, schema-drift shims and backstop migrations.
- The main structural risk is not the frontend compiling; it is that authorization, company context and workflow integrity still depend on drift-tolerant client behavior and repair-oriented database functions.
- Highest-risk operational surfaces: auth resolution, company/membership context, driver provisioning, direct client mutation of jobs/quotes/invoices/settings, and document/storage workflows.

2. EVIDENCE GATHERED

- All app/**/page.tsx routes statically inventoried and cross-referenced against imports, tables, RPCs and fetch calls.
- All app/components/*.tsx and lib/*.ts helpers inventoried.
- All app/api/**/*.ts routes inspected.
- All 54 Supabase migrations enumerated and late-stage repair migrations reviewed in depth.
- database/schema.sql compared against late migrations to identify schema-authority drift.
- Local validation executed: npm install, npm run typecheck, npm run lint, npm run build.
- Installed core package versions confirmed with npm ls: next 15.5.18, react 19.2.3, react-dom 19.2.3, @supabase/supabase-js 2.97.0.

3. GLOBAL ARCHITECTURAL FINDINGS

- Protected application routes are client-gated by app/components/ProtectedRoute.tsx; middleware.ts only injects CSP/security headers and no longer performs auth/authorization enforcement.
- AuthContext + authSession together form the real runtime access layer. They resolve profile, membership, driver and creator-company context after Supabase auth has already succeeded.
- authSession contains both authentication context resolution and data-repair behavior (auto-provision company, bootstrap membership, bootstrap missing profile). This is practical for recovery but dangerous as a permanent production invariant.
- Several pages import supabaseSchemaCompat or companySettings fallback logic, which is direct evidence that runtime schema drift has already been normalized into frontend behavior.
- High-value operational tables (jobs, quotes, invoices, settings, some documents) are still mutated directly from the client in many flows, leaving correctness heavily dependent on RLS alone.

4. ROUTE AND PAGE AUDIT

Protected-route global note:

Every protected page below depends on AuthContext hydration plus ProtectedRoute client redirects. Therefore every protected route has a shared refresh/logout/login persistence risk: stale or missing client session state can produce a blank/loading shell, late redirect to /login, or late redirect to /forbidden after the route has already mounted client-side.

/ (app/page.tsx)

- Purpose: public landing page that also client-redirects authenticated users to their canonical app area.
- User role: public.
- Auth requirement: no route auth; uses AuthContext to redirect signed-in users.
- Components used: LandingPage.
- Forms present: no direct form in this file.
- Data loaded: no direct Supabase reads in this file.
- Supabase tables touched: none.
- API routes used: none.
- Missing fields / drift indicators: no SSR/server auth gate; redirect depends on hydrated client auth state.
- Broken logic / persistence risks: refresh may briefly show marketing surface before redirect.
- Production readiness status: PARTIAL READY.

/login (app/login/page.tsx)

- Purpose: primary sign-in page with inline password-reset request UI.
- User role: public.
- Auth requirement: no route auth; uses AuthContext login/resetPassword and redirects authenticated users.
- Components used: Link.
- Forms present: HTML forms for sign-in and reset-email request.
- Data loaded: no direct table reads; auth via Supabase through AuthContext.
- Supabase tables touched: none directly.
- API routes used: none.
- Missing fields / drift indicators: no stronger password policy or server-side reset throttling evidenced here; only sessionStorage cooldown.
- Broken logic / persistence risks: redirect target is client-evaluated; recovery flow depends on browser auth signals.
- Production readiness status: PARTIAL READY.

/register (app/register/page.tsx)

- Purpose: public self-registration for customer or company accounts.
- User role: public.
- Auth requirement: none.
- Components used: Link, RegisterRole.
- Forms present: HTML registration form: email, account type, password, confirm password.
- Data loaded: no direct table reads; writes through supabase.auth.signUp metadata.
- Supabase tables touched: none directly.
- API routes used: none.
- Missing fields / drift indicators: no dedicated company profile capture at registration; no self-service driver registration; no password complexity beyond length 8.
- Broken logic / persistence risks: company onboarding is deferred to later auth/session repair logic.
- Production readiness status: PARTIAL READY.

/auth/callback (app/auth/callback/page.tsx)

- Purpose: consumes auth tokens/codes/OTP links and resolves the authenticated app user.

- User role: public transitional route.
- Auth requirement: relies on Supabase callback signals; redirects after user resolution.
- Components used: T.
- Forms present: none.
- Data loaded: indirectly triggers authSession resolution against profiles/company_memberships/drivers/companies.
- Supabase tables touched: indirect only.
- API routes used: none.
- Missing fields / drift indicators: no server-side fallback route; all handoff logic runs in the browser with timeout guards.
- Broken logic / persistence risks: expired links, missing RPCs, or slow auth resolution strand users on callback state.
- Production readiness status: PARTIAL READY.

/reset-password (app/reset-password/page.tsx)

- Purpose: completes invite/recovery password setup after Supabase session handoff.
- User role: public transitional route.
- Auth requirement: requires a valid password-setup session resolved client-side from Supabase auth.
- Components used: none exported from this page.
- Forms present: HTML form for new password and confirm password.
- Data loaded: no table reads; updates Supabase auth user password and signs out.
- Supabase tables touched: none directly.
- API routes used: none.
- Missing fields / drift indicators: only minimum length validation is present.
- Broken logic / persistence risks: entire flow depends on browser token handoff order and existing session recovery.
- Production readiness status: PARTIAL READY.

/forbidden (app/forbidden/page.tsx)

- Purpose: static access-denied page for failed ProtectedRoute authorization.
- User role: any redirected user.
- Auth requirement: none.
- Components used: Link.
- Forms present: none.
- Data loaded: none.
- Supabase tables touched: none.
- API routes used: none.
- Missing fields / drift indicators: no support/contact escalation path beyond generic navigation.
- Broken logic / persistence risks: authorization failures are explained only after client-side redirect.
- Production readiness status: READY.

/privacy (app/privacy/page.tsx)

- Purpose: static privacy policy page.
- User role: public.
- Auth requirement: none.
- Components used: Section.
- Forms present: none.
- Data loaded: static company config only.
- Supabase tables touched: none.
- API routes used: none.
- Missing fields / drift indicators: no operational DSAR/export/deletion workflow links are exposed.

- Broken logic / persistence risks: policy assertions may outpace implemented operational controls.
- Production readiness status: PARTIAL READY.

/terms (app/terms/page.tsx)

- Purpose: static terms and conditions page.
- User role: public.
- Auth requirement: none.
- Components used: Section.
- Forms present: none.
- Data loaded: static company config only.
- Supabase tables touched: none.
- API routes used: none.
- Missing fields / drift indicators: no workflow-specific commercial terms surfaced inside transactional flows.
- Broken logic / persistence risks: business terms exist, but static wording is not visibly bound to transactional UI.
- Production readiness status: PARTIAL READY.

/cookies (app/cookies/page.tsx)

- Purpose: static cookie policy page.
- User role: public.
- Auth requirement: none.
- Components used: Section.
- Forms present: none.
- Data loaded: static company config only.
- Supabase tables touched: none.
- API routes used: none.
- Missing fields / drift indicators: no cookie preference UI or consent-management surface exists in the app.
- Broken logic / persistence risks: policy says essential-only public cookies; runtime verification not performed.
- Production readiness status: PARTIAL READY.

/admin (app/admin/page.tsx)

- Purpose: company control-centre dashboard with KPI cards, finance/compliance/activity widgets and shortcuts.
- User role: admin/company/owner-equivalent.
- Auth requirement: ProtectedRoute with admin access expectations.
- Components used: PageHeader, StatCard, ActivityFeed, various card/table primitives.
- Forms present: no major create/edit form in the main dashboard page.
- Data loaded: companies, company_memberships context, jobs summary, invoices summary, quotes summary, vehicles, documents, dashboard counters, exchange/rate snapshot fallback.
- Supabase tables touched: companies, jobs, invoices, quotes, vehicles, documents, company_memberships, profiles.
- API routes used: none directly; reads via client Supabase.
- Missing fields / drift indicators: dashboard assumes company_id resolution succeeds; several widgets degrade silently on missing schema columns via compatibility helpers.
- Broken logic / persistence risks: stale role/company hydration can mis-state counts or expose empty dashboard.
- Production readiness status: PARTIAL READY.

/admin/drivers (app/admin/drivers/page.tsx)

- Purpose: driver management surface for create/list/invite/update state.
- User role: admin/company/owner-equivalent.
- Auth requirement: ProtectedRoute.

- Components used: PageHeader, tables, modal/form primitives.
- Forms present: add/edit driver form.
- Data loaded: drivers list excluding login_pin/device_token in select; company context.
- Supabase tables touched: drivers, company_memberships, profiles.
- API routes used: POST /api/admin/drivers.
- Missing fields / drift indicators: driver model still contains invite/password bootstrap complexity split across page, API route and reset-password flow.
- Broken logic / persistence risks: duplicate or partial driver creation possible if auth invite, profile upsert and driver row insert do not all succeed atomically.
- Production readiness status: PARTIAL READY.

/admin/fleet (app/admin/fleet/page.tsx)

- Purpose: fleet management surface for vehicles/documents overview.
- User role: admin/company/owner-equivalent.
- Auth requirement: ProtectedRoute.
- Components used: PageHeader, forms, tables, document/fleet widgets.
- Forms present: vehicle creation/edit, possibly maintenance/document input.
- Data loaded: vehicles, related documents, company settings context.
- Supabase tables touched: vehicles, documents, companies.
- API routes used: none directly observed.
- Missing fields / drift indicators: overlaps materially with /admin/drivers-vehicles.
- Broken logic / persistence risks: duplicate UI surfaces can diverge in validation rules and user expectations.
- Production readiness status: PARTIAL READY.

/admin/drivers-vehicles (app/admin/drivers-vehicles/page.tsx)

- Purpose: combined drivers and vehicles management surface.
- User role: admin/company/owner-equivalent.
- Auth requirement: ProtectedRoute.
- Components used: PageHeader, tabs/cards/tables.
- Forms present: driver and vehicle management forms.
- Data loaded: drivers, vehicles, company context.
- Supabase tables touched: drivers, vehicles, company_memberships, companies.
- API routes used: may indirectly overlap with /api/admin/drivers.
- Missing fields / drift indicators: strategic duplication with /admin/drivers and /admin/fleet.
- Broken logic / persistence risks: same entities managed in multiple admin surfaces increases mismatch risk.
- Production readiness status: PARTIAL READY.

/admin/diary (app/admin/diary/page.tsx)

- Purpose: diary/planning interface.
- User role: admin/company/owner-equivalent.
- Auth requirement: ProtectedRoute.
- Components used: diary/schedule widgets and header components.
- Forms present: schedule/job planning inputs.
- Data loaded: jobs/assignments/drivers/vehicles dates.
- Supabase tables touched: jobs, drivers, vehicles.
- API routes used: none directly observed.
- Missing fields / drift indicators: scheduling correctness depends on jobs schema and assignment semantics not strongly server-enforced.

- Broken logic / persistence risks: optimistic client updates can desync from actual job state.
- Production readiness status: PARTIAL READY.

/admin/jobs, /admin/jobs/[id], /admin/quotes, /admin/bids, /admin/invoices, /admin/documents, /admin/settings, /admin/customers, /admin/reports, /admin/compliance, /admin/driver-app, /admin/pod, /admin/tracking, /admin/notifications and other admin children

---

- Purpose: operational CRUD and reporting surfaces across jobs/quotes/bids/invoices/documents/settings/customers.
- User role: admin/company/owner-equivalent.
- Auth requirement: ProtectedRoute.
- Components used: repeated use of PageHeader, tables, forms, upload widgets, modal components and financial/status cards.
- Forms present: yes; these pages carry most platform business forms.
- Data loaded: jobs, quotes, bids, invoices, customers, documents, notifications, company settings, diary-style data.
- Supabase tables touched: jobs, quotes, quote_items, invoices, invoice_items, customers, documents, companies, company_memberships, drivers, vehicles and supporting lookup/config structures.
- API routes used: mostly direct client Supabase rather than dedicated server API.
- Missing fields / drift indicators: several flows rely on permissive client inserts/updates and compatibility fallbacks.
- Broken logic / persistence risks: without server-side transactional orchestration, partial saves and cross-table drift remain credible.
- Production readiness status: MIXED/PARTIAL; buildable, but not yet a hard-controlled enterprise surface.

/driver (app/driver/page.tsx)

- Purpose: driver dashboard/home for authenticated driver users.
- User role: driver.
- Auth requirement: ProtectedRoute driver access.
- Components used: dashboard cards, driver job list widgets.
- Forms present: limited direct forms on root; action buttons to workflow pages.
- Data loaded: driver row by user_id, jobs assigned to driver, possibly documents/POD status.
- Supabase tables touched: drivers, jobs, documents.
- API routes used: none directly on root.
- Missing fields / drift indicators: driver identity depends on drivers.user_id mapping being present and current.
- Broken logic / persistence risks: if user exists in auth but not in drivers row, route becomes unusable until repair.
- Production readiness status: PARTIAL READY.

/driver/jobs and /driver/jobs/[id]

- Purpose: assigned-job list and job detail for drivers.
- User role: driver.
- Auth requirement: ProtectedRoute driver access.
- Components used: job cards, status/action buttons, document upload or POD components.
- Forms present: status update, proof of delivery/upload fields depending on page.
- Data loaded: jobs scoped to driver, job documents, customer/location fields.
- Supabase tables touched: jobs, documents, possibly storage objects.
- API routes used: none directly observed.
- Missing fields / drift indicators: no evidence of strong server-side transition validation.
- Broken logic / persistence risks: client-side direct writes can allow invalid status transitions if RLS is permissive.
- Production readiness status: PARTIAL READY.

/driver/password

- Purpose: forced password change flow for invited drivers.
- User role: driver.
- Auth requirement: authenticated driver bearer session.
- Components used: form primitives.
- Forms present: new password form.
- Data loaded: none directly.
- Supabase tables touched: drivers flag update indirectly through API.
- API routes used: POST /api/driver/password.
- Missing fields / drift indicators: password policy limited to min length in API.
- Broken logic / persistence risks: relies on service-role admin password update path.
- Production readiness status: PARTIAL READY.

/customer and customer child routes

- Purpose: customer portal/dashboard, quotes, jobs, invoices and related tracking surfaces.
- User role: customer.
- Auth requirement: ProtectedRoute customer access.
- Components used: customer dashboard cards/tables/forms.
- Forms present: quote request and account-facing forms depending on child route.
- Data loaded: customer profile, quotes, jobs, invoices, documents relevant to the customer.
- Supabase tables touched: customers, quotes, jobs, invoices, documents, company/customer linkage tables.
- API routes used: none directly observed.
- Missing fields / drift indicators: customer/company scoping depends on profile and membership derivation being stable.
- Broken logic / persistence risks: weak isolation could leak operational or invoice data across customers if RLS is wrong.
- Production readiness status: PARTIAL READY.

/m, /m/jobs, /m/jobs/[id]

- Purpose: legacy mobile/admin convenience surface.
- User role: historically mobile/admin; current state is effectively deprecated for jobs routes.
- Auth requirement: route varies.
- Components used: lightweight mobile wrappers.
- Forms present: root may show navigation; jobs/detail pages are disabled placeholders.
- Data loaded: minimal.
- Supabase tables touched: negligible in current disabled state.
- API routes used: none.
- Missing fields / drift indicators: duplicate surface still exists in routing tree.
- Broken logic / persistence risks: users can discover dead-end legacy routes.
- Production readiness status: LEGACY / SHOULD BE ARCHIVED OR REDIRECTED CLEARLY.

ROUTE SUMMARY JUDGMENT

- Public/legal pages are straightforward.
- Auth transition pages are functional but still browser-session fragile.
- Admin surfaces are broad and feature-rich, but many critical workflows remain client-first rather than server-led.
- Driver/customer surfaces depend strongly on correct profile/membership/driver-row integrity.
- Legacy /m routes remain a cleanup target.

5. COMPONENT AUDIT

==========

Important app/components findings:

AuthContext.tsx

- File path: app/components/AuthContext.tsx
- Where used: app-wide provider from layout/client tree.
- Props/state: manages user, loading, auth session, reset state, company/profile/driver resolution side effects.
- Auth/data dependency: hard dependency on Supabase auth plus authSession and callback/recovery flow helpers.
- Risk level: HIGH.
- Risk reason: centralizes login/logout/reset/recovery/session hydration; failure here blocks the whole app.
- Legacy/duplicate status: active core component.

ProtectedRoute.tsx

- File path: app/components/ProtectedRoute.tsx
- Where used: protected admin/customer/driver pages.
- Props/state: allowedRoles, children, redirect state.
- Auth/data dependency: depends fully on AuthContext resolved app role.
- Risk level: HIGH.
- Risk reason: client-only gate means sensitive route decision happens after mount.
- Legacy/duplicate status: active core component.

PageHeader.tsx / dashboard card primitives / operational table wrappers

- Where used: extensively across admin/customer/driver pages.
- Props/state: presentational with some action callbacks.
- Auth/data dependency: usually indirect through page-level data.
- Risk level: LOW to MEDIUM.
- Legacy status: active shared UI.

PODPhotoUpload.tsx

- Where used: intended for proof-of-delivery image capture/upload.
- Props/state: file selection/upload callbacks.
- Auth/data dependency: storage bucket and job/document linkage.
- Risk level: MEDIUM.
- Legacy/duplicate status: static import scan suggested likely unused or narrowly used; requires confirmation before removal.

SignatureCanvas.tsx

- Where used: intended for signature capture flows.
- Props/state: signature drawing state.
- Auth/data dependency: POD/document workflow.
- Risk level: MEDIUM.
- Legacy/duplicate status: static import scan suggested possibly unused.

Toast.tsx / DelayUpdate.tsx

- Where used: low or no observed direct imports in current scan.
- Props/state: utility UI wrappers.
- Auth/data dependency: none/core UI only.
- Risk level: LOW.
- Legacy/duplicate status: likely dead or legacy, confirm before deletion.

Component-level conclusion

- Component library is not the primary risk area; the major risk is orchestration logic around auth, company context and

direct data mutation.

- Dead/legacy signal exists in a small set of utility/mobile/POD support components.

6. HOOK / HELPER / LIB AUDIT

lib/authSession.ts

- What it does: resolves the authoritative application context from a Supabase auth user; gathers profile, company_membership, driver and company rows; auto-provisions missing profile/company/membership in some cases.
- What depends on it: AuthContext, auth callback routing, post-login redirects, role gating.
- What can break: if any of the expected tables/functions are missing or drifted, login succeeds but app context fails; hidden repair side effects can create data that masks deeper schema problems.
- Audit verdict: HIGH RISK core file.

lib/authRole.ts

- What it does: maps raw role signals into app roles; current mapping treats admin_staff and company_admin as admin.
- What depends on it: ProtectedRoute and role-based routing decisions.
- What can break: any mismatch between database role vocabulary and frontend canonical mapping causes false denials or wrong-route redirects.
- Audit verdict: HIGH IMPACT, small file but critical semantics.

lib/authContextResolver.ts

- What it does: secondary helper for profile/company/driver resolution logic.
- What depends on it: auth/session bootstrap paths.
- What can break: duplicate role/company derivation logic across helpers creates inconsistent context.
- Audit verdict: MEDIUM-HIGH.

lib/authSessionPersistence.ts / authFlow.ts / authSession.ts adjacent helpers

---

- What they do: preserve auth/recovery state across browser transitions and normalize auth flow branches.
- What depends on them: login, recovery, callback, reset-password, logout stability.
- What can break: browser storage/callback ordering issues strand users in limbo states.
- Audit verdict: MEDIUM-HIGH because these files compensate for fragile browser auth transitions.

lib/activeCompany.ts

- What it does: resolves active company context for current user/session.
- What depends on it: most admin pages and any multi-company-sensitive queries.
- What can break: incorrect company resolution produces cross-company leakage or empty dashboards.
- Audit verdict: HIGH IMPACT.

lib/companySettings.ts

- What it does: loads/normalizes company configuration with drift-tolerant defaults.
- What depends on it: branding, settings, legal/company display, dashboards.
- What can break: missing settings columns or null rows cause degraded but hidden failure.
- Audit verdict: MEDIUM; also a schema-drift indicator.

lib/supabaseClient.ts

- What it does: shared browser Supabase client creation.
- What depends on it: nearly entire frontend.
- What can break: wrong env vars or project mismatch breaks all auth/data access.
- Audit verdict: HIGH IMPACT foundational helper.

lib/supabaseAdmin.ts (app/api/_lib/supabaseAdmin.ts)

- What it does: creates server-side service-role client for API routes.
- What depends on it: /api/admin/drivers, /api/driver/password and any future privileged routes.
- What can break: missing service-role secret disables privileged APIs; misuse bypasses RLS.
- Audit verdict: HIGH RISK by design; must stay extremely constrained.

lib/supabaseSchemaCompat.ts

- What it does: schema compatibility guards/fallback access patterns.
- What depends on it: pages operating across partially reconciled schema states.
- What can break: if overused, it normalizes drift indefinitely and hides missing migrations.
- Audit verdict: MEDIUM-HIGH; valuable short-term, but a long-term cleanup target.

Other lib helpers

- jobClientFields.ts: normalizes client-visible job fields; low-medium risk but can silently omit critical fields.
- invitation/session/recovery helpers: medium risk because they glue invite and password-setup lifecycle.
- formatting/date/finance helpers: low risk individually, but duplicated usage across invoices/jobs can produce UI drift.

Helper-level conclusion

The lib layer proves the platform has already needed significant runtime normalization. The largest concern is not a single bug; it is the concentration of auth + company + repair logic in a few foundational helpers.

7. API ROUTE AUDIT

/api/admin/drivers/route.ts

- Method: POST.
- Purpose: invite/create a driver user and related records.
- Auth validation: bearer token required; route resolves calling user through Supabase and checks membership/role.
- Payload validation: manual field checks only; no zod/schema validator despite zod dependency existing in package.json.
- Service-role usage: YES, via supabaseAdmin/service role to invite auth users and mutate privileged rows.
- Structural flow: verify caller -> verify company/admin privileges -> invite/update auth user -> upsert profile -> create/update driver row.
- Security risks: service role can bypass RLS; any missed company/role guard becomes high impact.
- Missing validation: stronger email/name/phone normalization, idempotency handling, duplicate-driver prevention, transactional rollback semantics.
- Response handling: JSON responses with explicit error messages; partial-failure paths can still leave orphaned auth or profile records.
- Production readiness: PARTIAL READY, HIGH-RISK PRIVILEGED ROUTE.

/api/driver/password/route.ts

- Method: POST.
- Purpose: lets authenticated driver set a new password and clears forced-change flag.
- Auth validation: bearer token required and current user fetched from Supabase.
- Payload validation: password length >= 8 only.
- Service-role usage: YES, updates auth user password with admin client and driver row flag.
- Security risks: minimal password policy; service-role path is powerful.
- Missing validation: password complexity/history/reuse, better error classification, rate limiting evidence.
- Response handling: simple JSON result.
- Production readiness: PARTIAL READY.

/app/auth/confirm/route.ts

- Method: GET/confirmation callback surface.
- Purpose: handles email confirmation links.
- Auth validation: based on Supabase confirmation parameters.
- Payload validation: query-string driven.
- Service-role usage: not primary.
- Security risks: mostly around correct token handoff and redirect target control.
- Missing validation: limited server-side hardening evidence.
- Response handling: redirect oriented.
- Production readiness: PARTIAL READY.

API summary

- The API layer is still thin. That keeps code surface smaller, but it also means many important business operations are not concentrated in auditable server transactions.
- Where server routes exist, they rely on service-role clients and manual validation, so they must be treated as privileged choke points and hardened further.

8. DATABASE / TABLE RELATIONSHIP AUDIT

==========

Tables clearly referenced by code and/or migrations:

profiles

- Expected columns/signals: id/user_id, email, role, full_name/name, company_id and profile metadata.
- Where used: authSession, AuthContext, admin/customer/driver context resolution.
- Relationships: auth.users -> profiles; profiles may link to companies/drivers/customers.
- RLS dependency: strong; profile data is foundational to role-based visibility.
- CRUD expectation: select on login, upsert on auth repair/invite flows.
- Risks: missing profile rows are actively backfilled; this proves profile presence is not guaranteed.

companies

- Expected columns/signals: id, name, slug, settings/branding/legal/contact fields and historical extra columns.
- Where used: dashboards, settings, legal/company display, membership resolution.
- Relationships: companies -> company_memberships, drivers, vehicles, jobs, invoices, quotes, documents.
- RLS dependency: essential for tenant isolation.
- CRUD expectation: select widely; created via registration/bootstrap path.
- Risks: authSession can create missing company for some roles; this is a major invariant-repair smell.

company_memberships

- Expected columns/signals: user_id, company_id, role_in_company, status.
- Where used: nearly all company-scoped admin resolution.
- Relationships: join table between users/profiles and companies.
- RLS dependency: critical.
- CRUD expectation: select on login, insert in bootstrap path, used to authorize admin routes.
- Risks: migration 050 inserts role_in_company='member' while schema enum in database/schema.sql lists owner/admin/dispatcher/viewer only. This is a confirmed schema drift contradiction.

drivers

- Expected columns/signals: id, user_id, company_id, email, full_name, phone, status, force_password_change, login_pin/device_token legacy-sensitive fields.
- Where used: admin driver management, driver portal, job assignment.

- Relationships: companies -> drivers; drivers -> jobs/documents.
- RLS dependency: driver self-access plus company admin access.
- CRUD expectation: select/update from admin and driver flows.
- Risks: invite/auth/profile/driver creation is multi-step and not fully transactional.

vehicles

- Expected columns/signals: id, company_id, registration, make/model/type/status/compliance metadata.
- Where used: fleet pages, diary/planning, jobs assignment.
- Relationships: companies -> vehicles; vehicles may relate to documents/jobs.
- RLS dependency: company-scoped isolation.
- Risks: overlapping admin surfaces increase inconsistent validation risk.

jobs

- Expected columns/signals: id, company_id, driver_id, customer_id, status, locations, scheduling and POD fields.
- Where used: admin jobs, driver jobs, customer views, diary dashboard.
- Relationships: companies/customers/drivers/vehicles/documents.
- RLS dependency: very high; operational core table.
- CRUD expectation: select/update/insert across many pages.
- Risks: many client-side direct writes; transition rules appear weakly server-enforced.

quotes / bids

- Expected columns/signals: quote header and line items, customer/company linkage, status and pricing fields.
- Where used: admin/customer quote workflows and bidding surfaces.
- Relationships: companies/customers and potentially jobs/invoices.
- RLS dependency: high.
- Risks: quote-to-job/invoice lifecycle integrity may drift without server transactions.

invoices / invoice_items

- Expected columns/signals: invoice number, company/customer/job references, totals, status, client_name and line items.
- Where used: admin finance surfaces, possibly customer portal.
- Relationships: companies/customers/jobs.
- RLS dependency: high because invoices carry commercially sensitive data.
- Risks: late migration 052 adding client_name is evidence of finance schema evolution after initial release.

documents

- Expected columns/signals: id, company_id, driver_id/vehicle_id/job_id, storage path/type/status/approval metadata.
- Where used: admin documents, POD flows, driver uploads.
- Relationships: companies/drivers/vehicles/jobs and storage.objects.
- RLS dependency: very high due uploaded personal/compliance material.
- Risks: storage policies depend on folder naming and auth_company_id helper; orphaned file vs row mismatch is credible.

customers

- Expected columns/signals: id, company_id, contact/billing fields.
- Where used: customer portal mapping, quotes/jobs/invoices.
- Relationships: companies -> customers -> quotes/jobs/invoices.
- RLS dependency: high to prevent cross-customer leakage.
- Risks: customer-role derivation must align with customer records and profile identity.

Database conclusion

- The tenant key set is company_id + membership/user mapping, not a single immutable server-issued tenant token.
- Any inconsistency across profiles, company_memberships, drivers or companies can destabilize route access and RLS expectations.
- Orphan risks exist for auth user/profile/membership/company bootstraps, and for document row vs storage object state.

9. MIGRATION CHAIN AUDIT (54 MIGRATIONS)

Observed chain characteristics:
- The chain is not a clean forward-only product evolution; it contains multiple repair, backstop, reconciliation and idempotent safety migrations.
- There are strong signals of historic schema drift and environment inconsistency.

Confirmed patterns

- Duplicated fixes / repeated hardening: late migrations repeatedly tighten RLS, reconcile schema, restore missing functions and add IF NOT EXISTS safety clauses.
- Contradictory schema authority: database/schema.sql is not fully authoritative; 039_schema_reconciliation.sql states runtime-critical functions were missing from the schema snapshot.
- Legacy patches: migrations such as complete schema/idempotent setup/backstop reconciliation indicate previous deploys were not consistently converged.
- Risky dependencies: auth/company context depends on database functions such as get_or_create_company_for_user, bootstrap_company_membership, auth_company_id and possibly helper functions restored by reconciliation migrations.
- Environment mismatch risk: managed Supabase restrictions forced use of public.auth_company_id() instead of a storage schema function. This is intentional but proves environment-specific adaptation.
- Migration order risk: storage, RLS and operational helper functions must exist in the correct sequence or route/auth behavior breaks in subtle ways.

High-confidence migration findings

- 032_storage_buckets.sql establishes storage buckets and policies using public.auth_company_id(); storage isolation is therefore dependent on folder/path conventions plus helper function correctness.
- 033 and 034 continue least-privilege tightening, especially around driver and operational data.
- 038_runtime_operational_rls_backstop.sql is itself a red-flag filename: it implies prior migrations were not enough.
- 039_schema_reconciliation.sql explicitly repairs missing runtime functions/columns and documents drift.
- 049 and 050 bootstrap company membership from auth context; this is functional but indicates onboarding invariants are still being repaired in-database.
- 050 conflicts with database/schema.sql by using role_in_company='member'.
- 052_invoices_client_name_guard.sql confirms finance schema continued to need safety guards late in the chain.

Migration verdict

- Migration chain is serviceable but fragile.
- A fresh environment may converge if the exact full chain is applied, yet the presence of reconciliation/backstop files means drift has historically been real and must be treated as a continuing production risk.

10. RLS / SECURITY AUDIT

==========

Key security posture findings:

Client-side vs server-side protection

- Middleware no longer performs route authorization; it only sets security headers/nonces.
- ProtectedRoute performs client-only access gating after the app bundle has loaded.
- Therefore security of actual data rests primarily on Supabase RLS and any server API role checks, not on Next route middleware.

Service-role usage risks

- app/api/_lib/supabaseAdmin.ts enables privileged service-role operations.
- Current audited privileged routes: /api/admin/drivers and /api/driver/password.
- This is acceptable only if route-specific authorization is airtight; any guard regression becomes full-privilege risk.

Public API / auth transition risks

- Auth callback, confirm and reset flows are heavily token/query/browser-session dependent.
- The platform has added significant compensation logic, which improves resilience but proves the flow was brittle.

User/company isolation

- Isolation depends on company_id alignment across profiles, company_memberships, drivers, customers, jobs, documents and storage folder conventions.
- Because authSession can repair missing company/membership/profile state, isolation assumptions are not guaranteed before login-time repair runs.

Role separation

- Admin/operator/company roles are normalized in frontend helpers.
- Driver/customer roles appear separated, but correctness depends on profile role vocabulary and driver/customer table alignment.
- Any drift between DB role values and authRole mapping can cause overgrant or overdeny behavior.

File upload / storage bucket risks

- Buckets observed: driver-docs, vehicle-docs, pod-photos.
- Policy model depends on helper function + path convention; that is workable but brittle.
- Risks: orphaned files, inconsistent path generation, document row present but storage object absent, storage object retained after row delete, and cross-company exposure if path or helper semantics drift.

Data leakage risks

- Highest leakage risk tables: invoices, documents, jobs, customers, drivers.
- Highest leakage mechanisms: permissive/misaligned RLS, wrong company_id assignment, wrong membership resolution, direct client selects using broad filters, and storage policy mistakes.

Security verdict

- Security model is conceptually sound only if RLS is correct everywhere.
- The platform still needs a full server-side invariants pass for privileged workflow writes and a simplified, authoritative tenant/role model.

11. FULL WORKFLOW AUDIT

Registration

- UI entry point: /register
- Data required: email, password, confirm password, account type.
- Payload path: supabase.auth.signUp with user metadata.
- Tables affected: auth.users initially; later profiles/companies/company_memberships through follow-up flows.
- Expected RLS: N/A at signup; later tenant scoping required.
- Expected result: account created and confirmation/recovery flow continues.
- Failure points: missing profile/company/membership rows after auth creation, inconsistent role metadata.
- Severity: HIGH.
- Repair recommendation: move onboarding invariant creation into a single server-led transaction.

Login

- UI entry point: /login
- Data required: email, password.
- Payload path: AuthContext login -> Supabase auth -> authSession context resolution.
- Tables affected: profiles, companies, company_memberships, drivers may be read/repaired.
- Failure points: auth ok but context repair fails; slow hydration; wrong route mapping.
- Severity: HIGH.
- Repair recommendation: split login from data repair and surface explicit remediation states.

Password reset / invite completion

- UI entry point: /login reset form, email links, /auth/callback, /reset-password, /driver/password.
- Data required: email or reset token; new password.
- Payload path: Supabase recovery -> callback handoff -> password update or /api/driver/password.
- Tables affected: auth.users, drivers.force_password_change.
- Failure points: broken token handoff, expired links, partial invite state.
- Severity: HIGH.
- Repair recommendation: unify invite/recovery states and strengthen password policy.

Company creation / membership creation

- UI entry point: implicit after registration/login for company-capable roles.
- Payload path: authSession / RPC bootstrap functions.
- Tables affected: companies, company_memberships, profiles.
- Failure points: company created without valid membership, membership role vocabulary drift.
- Severity: CRITICAL.
- Repair recommendation: make onboarding explicit and transactional, not login-time repair.

Driver creation / driver login / forced password change

- UI entry point: /admin/drivers -> /api/admin/drivers -> email invite -> reset/password pages.
- Tables affected: auth.users, profiles, drivers.
- Failure points: orphan auth invite, duplicate driver rows, missing driver.user_id, weak password validation.
- Severity: HIGH.
- Repair recommendation: transactional server workflow with idempotent invite reconciliation.

Vehicle creation / fleet management

- UI entry point: /admin/fleet and /admin/drivers-vehicles.
- Tables affected: vehicles, documents.
- Failure points: duplicate admin surfaces, inconsistent validations, document linkage errors.
- Severity: MEDIUM-HIGH.
- Repair recommendation: consolidate fleet surfaces and centralize validation.

Document upload / approval-rejection / POD

- UI entry point: admin document pages, driver job pages, POD/photo/signature components.
- Tables affected: documents, jobs, storage.objects.
- Failure points: object/row mismatch, bucket path mismatch, approval status drift.
- Severity: HIGH.
- Repair recommendation: server-issued upload intents and authoritative document transition rules.

Jobs / quotes / bids / diary

- UI entry point: admin operational pages, customer portal, driver job pages.

- Tables affected: jobs, quotes, bids and related records.
- Failure points: client-side direct writes, non-transactional state transitions, duplicate status logic across UIs.
- Severity: HIGH.
- Repair recommendation: server-side workflow commands for status transitions and cross-table changes.

Invoices

- UI entry point: /admin/invoices and customer invoice views.
- Tables affected: invoices, invoice_items, customers/jobs relations.
- Failure points: schema drift (client_name guard), totals integrity, tenant leakage.
- Severity: HIGH.
- Repair recommendation: server-side invoice creation/update with strict schema validation.

Settings / dashboard KPIs / customer portal / mobile /m routes

- Failure points: drift-tolerant settings layer, stale KPI reads, duplicate legacy mobile routes, company context ambiguity.
- Severity: MEDIUM to HIGH depending on surface.
- Repair recommendation: remove legacy routes and tighten settings/data contracts.

12. DEPENDENCY / PACKAGE / BUILD AUDIT

==========

package.json findings

- Build scripts present and functional in current repo state: lint, typecheck, build.
- Declared major stack: Next.js ^15.1.6, React ^19.2.0, React DOM ^19.2.0, Supabase JS ^2.97.0, TypeScript ~5.9.3.
- Installed versions resolved locally to newer compatible patch/minor releases, including next 15.5.18 and react 19.2.3.
- zod is declared but audited API routes still use manual payload validation.

Unused/maybe-unused dependency signals

- Static import scan suggested some declared UI/helper packages may be underused.
- Static scan also suggested likely-unused local components such as DelayUpdate.tsx, PODPhotoUpload.tsx, SignatureCanvas.tsx, Toast.tsx.
- This is only a static signal, not definitive runtime proof.

Build-risk conclusion

- Current repo builds locally.
- Primary build risk is environment configuration, not TypeScript correctness in the current branch.
- The previous 4506 TypeScript errors likely came from one of:

1) an older repository state,
2) missing node_modules / incorrect install,
3) a broken editor/sandbox cache,
4) a different tsconfig or branch in an earlier session.

- In the current repository snapshot, those errors are not reproducible.

13. DEPLOYMENT / NETLIFY AUDIT

netlify.toml / deployment findings

- Build command and environment validation are explicitly configured.
- scripts/validate-supabase-env.mjs enforces a valid NEXT_PUBLIC_SUPABASE_URL and anon key shape.
- netlify.toml also pins Node version behavior and uses the Netlify Next plugin.
- Publish behavior is consistent with Next.js deployment rather than a static-only export.

Deployment risks

- Production deploy remains highly sensitive to correct Supabase environment pairing.
- If env vars point at the wrong project, the app can compile successfully while auth/data semantics are wrong.
- Because route auth is client-side, a deployment can appear visually healthy before deeper data/RLS issues emerge.
- No live Netlify or production env verification was performed in this sandbox.

Deployment verdict

- Deployment config is not obviously broken in-source.
- Real risk lies in environment alignment, secret correctness and live RLS/storage state.

14. DEAD CODE / DUPLICATE / LEGACY AUDIT

Confirmed or strong-signal findings

- /m/jobs and /m/jobs/[id] are intentionally disabled legacy placeholders redirecting users elsewhere.
- /admin/fleet and /admin/drivers-vehicles overlap heavily and should be rationalized.
- app/components utility set includes likely-unused or legacy items requiring confirmation before deletion: DelayUpdate.tsx, PODPhotoUpload.tsx, SignatureCanvas.tsx, Toast.tsx.
- Old markdown/static reports exist in the repo history/context and should not be treated as runtime truth.
- supabaseSchemaCompat and related guards are useful today but represent schema-repair legacy surface.

Archival/removal candidates after confirmation

- Duplicate mobile/admin route surfaces under /m.
- Overlapping admin fleet/drivers-vehicles views.
- Compatibility shims once schema is fully reconciled.
- Likely-unused POD utility components if no runtime references remain.

15. UX / FUNCTIONALITY AUDIT

High-level UX blockers

- Multiple workflows depend on silent background auth/company repair, which is confusing when a user simply expects login to succeed or fail cleanly.
- Duplicate admin surfaces dilute discoverability and create uncertainty about the canonical place to manage vehicles and driver-vehicle relationships.
- Legacy /m routes create dead-end/mobile confusion.
- Empty/error/loading states are still dependent on client hydration, so users can see brief ambiguous states after refresh.
- Several operational pages appear rich in data tables/forms but lack obvious hardened edit/delete confirmation and failure-recovery orchestration at the server boundary.
- Mobile/responsive coverage exists in some surfaces, but no runtime verification was performed.

16. GDPR / LEGAL / BUSINESS READINESS AUDIT

What exists

- Privacy, Terms and Cookies pages exist as public legal surfaces.
- The app handles driver/customer/company/invoice/document data, which is clearly personal and commercially sensitive.

Business/legal gaps and risks

- No audited evidence of explicit retention policy workflows, DSAR/export flow, account deletion workflow, or document retention automation surfaced in the UI.

- Uploaded documents likely include compliance and personal identity data; storage/document lifecycle governance needs to be explicit, not just policy-text based.
- Customer and driver data isolation relies on technical controls whose live state was not verified here.
- Cookies policy exists, but no cookie preference/consent-management UI was observed.

Legal verdict

- Public policy pages help, but operational privacy/compliance readiness cannot be considered complete from the current code alone.

17. DETAILED REPAIR PLAN

==========

Phase 1 - Production blockers

---

1) Issue: Auth resolution coupled with data repair

- Severity: CRITICAL
- Affected files: lib/authSession.ts, app/components/AuthContext.tsx, app/auth/callback/page.tsx
- Root cause: login path also provisions missing company/membership/profile state.
- Why it matters: authentication success does not mean tenant context integrity already exists.
- Exact repair direction: split auth/session resolution from onboarding repair; fail explicitly when invariants are absent.
- Must test after repair: signup -> login -> refresh -> logout -> login -> callback -> reset-password.

2) Issue: Client-side route protection is the primary gate

- Severity: HIGH
- Affected files: app/components/ProtectedRoute.tsx, middleware.ts, protected pages.
- Root cause: middleware removed runtime auth enforcement; route decisions are made in the browser.
- Why it matters: access UX is delayed and sensitive pages depend entirely on RLS for true protection.
- Exact repair direction: add authoritative server-side gating patterns where feasible and reduce protected content pre-mount.
- Must test after repair: direct URL hits, refresh on protected routes, unauthorized role navigation.

Phase 2 - Auth / company / membership / isolation

3) Issue: company_memberships role vocabulary drift

- Severity: CRITICAL
- Affected files: database/schema.sql, supabase/migrations/050_bootstrap_company_membership.sql, role mapping helpers.
- Root cause: schema snapshot and migration logic disagree on allowed role values.
- Why it matters: onboarding and authorization can fail or diverge by environment.
- Exact repair direction: choose one canonical role set and reconcile schema, migrations, functions and frontend mapping.
- Must test after repair: fresh onboarding, existing-user login, admin access, non-admin denial.

4) Issue: Role/company resolution distributed across multiple helpers

- Severity: HIGH
- Affected files: lib/authRole.ts, lib/authContextResolver.ts, lib/activeCompany.ts, lib/authSession.ts
- Root cause: incremental evolution created multiple interpretation layers.
- Why it matters: inconsistent context increases overgrant/overdeny risk.
- Exact repair direction: centralize authoritative role + company resolution contract.
- Must test after repair: admin/company/driver/customer journeys and edge cases with missing rows.

Phase 3 - Broken workflows

5) Issue: Driver invite/create flow is not fully transactional

- Severity: HIGH
- Affected files: app/admin/drivers/page.tsx, app/api/admin/drivers/route.ts, driver reset flows.

- Root cause: auth invite, profile upsert and driver insert happen as chained steps.
- Why it matters: orphaned records and partial user onboarding are possible.
- Exact repair direction: move to idempotent server workflow with rollback/reconciliation strategy.
- Must test after repair: new driver, reinvite existing driver, failed partial create, forced password change.

6) Issue: Core jobs/quotes/invoices transitions are client-heavy
- Severity: HIGH
- Affected files: major admin/customer/driver operational pages.
- Root cause: many business writes happen directly from client Supabase calls.
- Why it matters: transition rules and cross-table integrity are weakly enforced.
- Exact repair direction: create server-side command endpoints or RPCs for workflow transitions.
- Must test after repair: quote->job, job->invoice, status transitions, customer visibility.

7) Issue: Document/storage workflow integrity is brittle
- Severity: HIGH
- Affected files: document pages/components, storage migrations, any POD upload surface.
- Root cause: row/object lifecycle and path-policy conventions are loosely coupled.
- Why it matters: data loss, orphan files and leakage risk.
- Exact repair direction: server-issued upload intents, explicit document state machine, cleanup/reconciliation tasks.
- Must test after repair: upload, replace, approve, reject, delete, cross-company isolation.

Phase 4 - Database / RLS / migration cleanup

8) Issue: Migration chain contains backstops and reconciliation patches
- Severity: HIGH
- Affected files: numerous migrations, database/schema.sql
- Root cause: historic schema drift and uneven deploy convergence.
- Why it matters: fresh environment reliability and auditability suffer.
- Exact repair direction: produce an authoritative schema baseline, reconcile role enums/functions/policies, and verify a clean bootstrap from zero.
- Must test after repair: clean Supabase project apply-all-migrations, smoke CRUD under each role.

9) Issue: Compatibility shims mask missing schema invariants
- Severity: MEDIUM-HIGH
- Affected files: lib/companySettings.ts, lib/supabaseSchemaCompat.ts and dependent pages.
- Root cause: runtime code was adapted to survive inconsistent database states.
- Why it matters: true schema defects remain hidden.
- Exact repair direction: eliminate shims after DB convergence and replace with strict contracts.
- Must test after repair: settings/dashboard pages against clean migrated schema.

Phase 5 - UX and missing functionality

10) Issue: Duplicate admin surfaces and ambiguous canonical workflows
- Severity: MEDIUM
- Affected files: /admin/fleet, /admin/drivers-vehicles, legacy /m pages.
- Root cause: iterative product growth without consolidation.
- Why it matters: operator confusion and inconsistent data entry.
- Exact repair direction: choose one canonical surface per domain and redirect/archive the others.
- Must test after repair: admin navigation, create/edit/search across fleet/driver-vehicle workflows.

11) Issue: Legal/compliance UX not operationalized
- Severity: MEDIUM-HIGH
- Affected files: privacy/terms/cookies pages and account/document settings surfaces.

- Root cause: legal pages exist without visible operational workflows for retention/consent/export/deletion.
- Why it matters: business readiness and GDPR defensibility remain incomplete.
- Exact repair direction: add operational privacy controls and retention governance.
- Must test after repair: user-facing policy access, export/delete request path, document retention handling.

Phase 6 - Dead code cleanup

12) Issue: Legacy routes/components remain in tree
- Severity: MEDIUM
- Affected files: /m routes, likely-unused POD/utility components, compatibility helpers.
- Root cause: previous implementations not fully retired.
- Why it matters: maintenance burden and accidental reuse of obsolete surfaces.
- Exact repair direction: confirm usage, archive/delete dead files, preserve only canonical paths.
- Must test after repair: route map, imports, production build, navigation links.

Phase 7 - Deployment and final production verification

13) Issue: Build success does not prove environment correctness
- Severity: HIGH
- Affected files: netlify.toml, scripts/validate-supabase-env.mjs, env secrets, live Supabase project.
- Root cause: live correctness depends on exact env, RLS, storage and migration state.
- Why it matters: production may fail in ways static audit cannot see.
- Exact repair direction: perform staged end-to-end verification on the target Supabase/Netlify environment after all schema and workflow repairs.
- Must test after repair: all auth roles, all business workflows, storage, legal access and error logging.

18. MANUAL TESTING PLAN AFTER FIXES

- Public pages: home, privacy, terms, cookies load and navigate correctly.
- Registration: customer and company registrations create consistent downstream profile/company/membership state.
- Login/logout/refresh: each role lands on correct route before and after browser refresh.
- Password recovery/invite: callback and reset flows succeed with valid links and fail cleanly with expired links.
- Admin: create/edit/archive driver, vehicle, customer, job, quote, invoice.
- Driver: first login, forced password change, assigned job view, POD upload/signature/photo.
- Customer: login, quote visibility, invoice visibility, document visibility restricted to own records.
- Storage: upload/replace/delete documents, verify path isolation and row/object consistency.
- RLS: cross-company and cross-customer access denial checks under real seeded accounts.
- Deployment: fresh build, correct env validation, no client-side auth redirect loops.

19. DATABASE VERIFICATION CHECKLIST

- Confirm every migration applies cleanly in order to a fresh Supabase project.
- Confirm canonical enum/value sets for profile role and company membership role.
- Confirm required helper functions exist exactly once with expected signatures.
- Confirm RLS enabled on every tenant-sensitive table.
- Confirm storage buckets and policies match document path conventions.
- Confirm no orphan auth.users / profiles / companies / company_memberships / drivers rows remain.
- Confirm invoices schema includes all fields used by current UI.
- Confirm company_id/user_id/membership_id constraints match code assumptions.

20. FINAL PRODUCTION READINESS VERDICT

Verdict: NOT YET PRODUCTION-READY AS A FULLY TRUSTED PLATFORM OF RECORD.

Reason:

- The repository now compiles and builds locally, which is positive.

- But the platform still carries structural auth/company/membership repair logic, schema drift history, migration contradictions, thin privileged APIs with manual validation, and storage/document workflow brittleness.

- Production readiness should only be reconsidered after the repair phases above and a real staged runtime verification pass.

RUNTIME CLASSIFICATION FOR THIS AUDIT

- All route/component/API/database/security/deployment findings in this document are STATICALLY VERIFIED FROM SOURCE.

- No live production/staging browser or database runtime claim is made here unless separately stated.

- Overall runtime status label for this report: NOT RUNTIME TESTED.

21. APPENDIX A - PAGE INVENTORY

- /admin/bids | file=app/admin/bids/page.tsx | tables=job_bids | api=- | forms=no | auth=protected

- /admin/companies | file=app/admin/companies/page.tsx | tables=companies,company_memberships,profiles | api=- | forms=no | auth=protected

- /admin/diary | file=app/admin/diary/page.tsx | tables=jobs | api=- | forms=no | auth=protected

- /admin/documents | file=app/admin/documents/page.tsx | tables=driver_documents,drivers,vehicle_documents,vehicles | api=- | forms=no | auth=protected

- /admin/drivers-vehicles | file=app/admin/drivers-vehicles/page.tsx | tables=driver_locations,drivers,vehicles | api=- | forms=no | auth=protected

- /admin/drivers | file=app/admin/drivers/page.tsx | tables=companies,drivers | api=/api/admin/drivers | forms=no | auth=protected

- /admin/fleet | file=app/admin/fleet/page.tsx | tables=driver_locations,drivers,vehicles | api=- | forms=no | auth=protected

- /admin/invoices/[id] | file=app/admin/invoices/[id]/page.tsx | tables=invoices | api=- | forms=no | auth=protected

- /admin/invoices | file=app/admin/invoices/page.tsx | tables=invoices | api=- | forms=no | auth=protected

- /admin/jobs/[id] | file=app/admin/jobs/[id]/page.tsx | tables=drivers,jobs | api=- | forms=no | auth=protected

- /admin/jobs | file=app/admin/jobs/page.tsx | tables=jobs | api=- | forms=no | auth=protected

- /admin | file=app/admin/page.tsx | tables=driver_documents,drivers,invoices,job_bids,jobs,quotes,vehicle_documents,vehicles | api=- | forms=no | auth=protected

- /admin/quotes | file=app/admin/quotes/page.tsx | tables=companies,company_memberships,jobs,quotes | api=- | forms=no | auth=protected

- /admin/settings | file=app/admin/settings/page.tsx | tables=companies,company_settings | api=- | forms=no | auth=protected

- /admin/vehicles | file=app/admin/vehicles/page.tsx | tables=companies,drivers,vehicles | api=- | forms=no | auth=protected

- /auth/callback | file=app/auth/callback/page.tsx | tables=- | api=- | forms=no | auth=protected

- /cookies | file=app/cookies/page.tsx | tables=- | api=- | forms=no | auth=public/mixed

- /customer | file=app/customer/page.tsx | tables=company_memberships,quotes | api=- | forms=no | auth=protected

- /driver/change-password | file=app/driver/change-password/page.tsx | tables=- | api=/api/driver/password | forms=yes | auth=protected

- /driver/jobs/[id] | file=app/driver/jobs/[id]/page.tsx | tables=jobs | api=- | forms=no | auth=protected

- /driver/jobs | file=app/driver/jobs/page.tsx | tables=drivers,jobs | api=- | forms=no | auth=protected

- /driver | file=app/driver/page.tsx | tables=drivers,jobs,vehicles | api=- | forms=no | auth=protected

- /forbidden | file=app/forbidden/page.tsx | tables=- | api=- | forms=no | auth=public/mixed

- /login | file=app/login/page.tsx | tables=- | api=- | forms=yes | auth=protected

- /m/jobs/[id] | file=app/m/jobs/[id]/page.tsx | tables=- | api=- | forms=no | auth=protected

- /m/jobs | file=app/m/jobs/page.tsx | tables=- | api=- | forms=no | auth=protected

- /m | file=app/m/page.tsx | tables=- | api=- | forms=no | auth=protected

- / | file=app/page.tsx | tables=- | api=- | forms=no | auth=protected

- /privacy | file=app/privacy/page.tsx | tables=- | api=- | forms=no | auth=public/mixed

- /register | file=app/register/page.tsx | tables=- | api=- | forms=yes | auth=protected

- /reset-password | file=app/reset-password/page.tsx | tables=- | api=- | forms=yes | auth=protected

- /terms | file=app/terms/page.tsx | tables=- | api=- | forms=no | auth=public/mixed

22. APPENDIX B - API INVENTORY

- file=app/api/admin/drivers/route.ts | tables=company_memberships,drivers,profiles | auth=yes

- file=app/api/driver/password/route.ts | tables=drivers | auth=yes

- file=app/auth/confirm/route.ts | tables=- | auth=mixed

23. APPENDIX C - COMPONENT INVENTORY

- file=app/components/AuthContext.tsx | lines=371 | forms=no | auth=yes

- file=app/components/DelayUpdate.tsx | lines=170 | forms=no | auth=no

- file=app/components/InvoiceTemplate.tsx | lines=433 | forms=no | auth=no

- file=app/components/LoginModal.tsx | lines=350 | forms=yes | auth=yes

- file=app/components/PODPhotoUpload.tsx | lines=168 | forms=no | auth=no

- file=app/components/ProtectedRoute.tsx | lines=68 | forms=no | auth=yes

- file=app/components/ServiceWorkerRegistration.tsx | lines=18 | forms=no | auth=no

- file=app/components/SignatureCanvas.tsx | lines=266 | forms=no | auth=no

- file=app/components/Toast.tsx | lines=63 | forms=no | auth=no

24. APPENDIX D - LIB INVENTORY

- file=lib/activeCompany.ts | tables=- | rpc=- | auth=yes

- file=lib/authContextResolver.ts | tables=- | rpc=- | auth=no

- file=lib/authFlow.ts | tables=- | rpc=- | auth=no

- file=lib/authRole.ts | tables=- | rpc=- | auth=no

- file=lib/authSession.ts | tables=companies,company_memberships,drivers,profiles | rpc=bootstrap_company_membership,get_or_create_company_for_user | auth=no

- file=lib/companySettings.ts | tables=companies,company_settings | rpc=- | auth=no

- file=lib/jobClientFields.ts | tables=- | rpc=- | auth=no

- file=lib/runtimeProof.ts | tables=- | rpc=- | auth=no

- file=lib/supabaseClient.ts | tables=- | rpc=- | auth=yes

- file=lib/supabaseSchemaCompat.ts | tables=- | rpc=- | auth=no

- file=lib/types/database.ts | tables=- | rpc=- | auth=no